

A Compiler Parameterized by a High-Level C Semantics

Staged compiler generation for an AArch64 C variant

16 August 2026

Abstract

This note makes the compiler itself a parameter of a high-level semantic description of a C variant. The description records integer width, entry symbol, result register, syscall-number register, syscall number, and syscall instruction. Source expressions are normalized before lowering. The description can be quoted as typed code and unquoted to generate the compiler, demonstrating staged self-application at the compiler level. Direct and staged compilation are compared byte-for-byte.

1 Semantic parameter

The compiler parameter is a record

$\text{CDescription} = \{\text{name} : \text{String}, \text{integerBits} : \text{Nat}, \text{entry} : \text{String}, \text{resultRegister} : \text{String}, \text{syscallRegister} : \text{String}, \text{write}$

It is high-level semantic data, not an assembly fragment. A compiler is a function

$$\text{compile} : \text{CDescription} \rightarrow \text{Expr}_C \rightarrow \text{Asm}_{AArch64}.$$

Changing the description changes target policy while leaving the source expression and compiler algorithm unchanged.

2 Source and normalization

The demonstration source language contains integer literals, addition, and an explicit write operation. The normalizer recursively folds additions whose operands are literals. Hence

$$40 + (2 + 1) \mapsto 43.$$

The compiler lowers the normalized term under the supplied description. This is partial evaluation: static source computation is performed before residual target code is emitted.

3 Staged compiler generation

The semantic description has a typed code boundary:

$$\text{quoteDescription} : D \rightarrow \text{Code}(D), \quad \text{unquoteDescription} : \text{Code}(D) \rightarrow D.$$

The staged generator is

$$\text{compileStaged}(\hat{D}, e) = \text{compile}(\text{unquoteDescription}(\hat{D}), e).$$

Because the quoted description is inert until unquoted, this is self-application of the compiler generator at an explicit stage, not an unrestricted $\text{Compiler} : \text{Compiler} \rightarrow \text{Compiler}$ inhabitant.

4 Evidence

The implementation is `parametric_c.cpp`. It was built with:

```
g++ -std=c++23 -Wall -Wextra -pedantic -O2 parametric_c.cpp -o parametric_c
./parametric_c
```

The output begins:

```
staged compiler equivalence: PASS
.text
.global main
main:
    stp x29, x30, [sp, #-16]!
    mov x29, sp
    mov x0, #43
    ldp x29, x30, [sp], #16
    ret
alternate C variant entry: entry
```

Direct compilation from the Linux C description and compilation through its quoted description produce identical assembly. A second description changes the entry symbol to ‘entry’, showing that target policy is genuinely parameterized.

5 Relation to the typed C subset

The full typed subset can use the same pattern with a richer description containing the type interpretation, calling convention, object layout, effect set, and lowering rules. The invariant becomes

$$\Gamma \vdash_D e : \tau \implies \Gamma \vdash_D \text{normalize}_D(e) : \tau \implies \text{emit}_D(e) : \text{Asm}_D(\tau).$$

The current file demonstrates the staging and parameterization boundary on a minimal expression fragment; the existing `typed_subset.cpp` supplies the richer typed AST and ABI example.

6 Limitations

The parameter record is intentionally small and does not yet describe register allocation, aggregate layout, or all typed-subset operators. Extending it should preserve typed code boundaries and make each new ABI choice explicit rather than embedding target policy in the compiler algorithm.